Informe Triggers Módulo 1

Usuario Roles y Acceso

Versión: 1.1

25 de abril de 2026

Juan David Peña Roldan

Universidad Surcolombiana

Neiva, Huila

2026

TRG-01 — Unicidad de contraseña (no reutilización)

Descripción	Impide que un usuario establezca una contraseña cuyo hash ya existe como su contraseña actual.
Aplicado a	RF-07 (Cambio de contraseña), RF-09 (Restablecimiento)
Restricción origen RF	"No se permite reutilizar la contraseña actual"
Justificación	Si el backend falla o se hace un UPDATE directo, la BD no puede comparar hashes (eso es lógica de bcrypt en backend). Este trigger es una **red de seguridad de último recurso**: valida que el campo contraseña_cifrada no sea idéntico al valor previo. No es sustituto de la lógica bcrypt del backend.

```
CREATE OR REPLACE FUNCTION
modulo1.trg_fn_no_reutilizar_contrasena()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.contrasena_cifrada = OLD.contrasena_cifrada THEN
        RAISE EXCEPTION 'CONSTRAINT_VIOLATION: La nueva contraseña
no puede ser idéntica a la anterior.'
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_reutilizar_contrasena
BEFORE UPDATE OF contrasena_cifrada ON modulo1.usuarios
FOR EACH ROW
WHEN (OLD.contrasena_cifrada IS NOT NULL)
EXECUTE FUNCTION modulo1.trg_fn_no_reutilizar_contrasena();
```

TRG-02 — Control de versión optimista (OCC)

Descripción	Incrementa automáticamente el campo version en cada UPDATE sobre usuarios, garantizando que el control de concurrencia optimista funcione incluso si el backend olvida hacerlo.
Aplicado a	RF-05 (Edición de datos)
Restricción origen RF	"El sistema debe aplicar control de concurrencia optimista para evitar conflictos de edición simultánea."
Justificación	El campo version ya existe en el esquema. Este trigger garantiza que nunca quede desincronizado, independientemente del origen del UPDATE.

```
CREATE OR REPLACE FUNCTION modulo1.trg_fn_incrementar_version()
RETURNS TRIGGER AS $$
BEGIN
    NEW.version := OLD.version + 1;
    NEW.fecha_actualizacion := now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_incrementar_version
BEFORE UPDATE ON modulo1.usuarios
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_incrementar_version();
```

TRG-03 — Protección de registros de auditoría (inmutabilidad)

Descripción	Bloquea cualquier intento de UPDATE o

	DELETE sobre la tabla eventos. Los registros de auditoría son inmutables por diseño.
Aplicado a	RF-10 (Historial de auditoría)
Restricción origen RF	"Los registros de auditoría son inmutables. No se deben permitir operaciones de UPDATE o DELETE sobre esta tabla, ni siquiera para el rol Administrador."
Justificación	Esta es la garantía más importante de la auditoría. No puede depender del backend ni de permisos de rol; debe ser una restricción incondicional a nivel de datos.

```
CREATE OR REPLACE FUNCTION modulo1.trg_fn_proteger_auditoria()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'IMMUTABLE_RECORD: Los registros de auditoría no
pueden ser modificados ni eliminados. Operación: %', TG_OP
        USING ERRCODE = 'P0002';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_proteger_auditoria_update
BEFORE UPDATE ON modulo1.eventos
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_proteger_auditoria();

CREATE TRIGGER trg_proteger_auditoria_delete
BEFORE DELETE ON modulo1.eventos
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_proteger_auditoria();
```

TRG-04 — Transiciones de estado de cuenta válidas

Descripción	Valida que el cambio de id_estado_cuenta en cuentas_usuarios respete la matriz de transiciones permitidas. Impide transiciones inválidas, especialmente desde el estado ELIMINADO (irreversible).
Aplicado a	RF-06 (Gestión de cuentas)
Restricción origen RF	*"El sistema impide transiciones de estado no definidas" / "Una cuenta en estado ELIMINADO tiene carácter irreversible"*
Justificación	La integridad del ciclo de vida de la cuenta no puede depender únicamente del backend. Un UPDATE directo a la tabla podría reactivar una cuenta eliminada.

```
CREATE OR REPLACE FUNCTION modulo1.trg_fn_validar_transicion_estado()
RETURNS TRIGGER AS $$
DECLARE
    v_estado_origen  VARCHAR(55);
    v_estado_destino VARCHAR(55);
BEGIN
    IF NEW.id_estado_cuenta = OLD.id_estado_cuenta THEN
        RETURN NEW;
    END IF;

    SELECT nombre INTO v_estado_origen
    FROM modulo1.estados_cuentas WHERE id_estado_cuenta =
OLD.id_estado_cuenta;

    SELECT nombre INTO v_estado_destino
    FROM modulo1.estados_cuentas WHERE id_estado_cuenta =
NEW.id_estado_cuenta;

    -- Estado ELIMINADO es irreversible
    IF v_estado_origen = 'ELIMINADO' THEN
        RAISE EXCEPTION 'INVALID_TRANSITION: Una cuenta en estado
ELIMINADO no puede cambiar a %. Esta transición es irreversible.',
v_estado_destino
            USING ERRCODE = 'P0003';
    END IF;
```

```
    -- Matriz de transiciones permitidas
    IF NOT (
        (v_estado_origen = 'PENDIENTE_ACTIVACION' AND v_estado_destino
IN ('ACTIVO', 'ELIMINADO'))
        OR (v_estado_origen = 'ACTIVO'             AND v_estado_destino
IN ('INACTIVO', 'BLOQUEADO', 'ELIMINADO'))
        OR (v_estado_origen = 'INACTIVO'           AND v_estado_destino
IN ('ACTIVO', 'ELIMINADO'))
        OR (v_estado_origen = 'BLOQUEADO'          AND v_estado_destino
IN ('ACTIVO', 'INACTIVO', 'ELIMINADO'))
    ) THEN
        RAISE EXCEPTION 'INVALID_TRANSITION: Transición de % a % no está
permitida por el modelo de estados.', v_estado_origen, v_estado_destino
            USING ERRCODE = 'P0003';
    END IF;

    NEW.fecha_cambio_estado := now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_transicion_estado
BEFORE UPDATE OF id_estado_cuenta ON modulo1.cuentas_usuarios
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_validar_transicion_estado();
```

TRG-05 — Protección del rol Administrador (no eliminación / no renombramiento)

Descripción	Bloquea DELETE y el cambio de nombre_rol sobre el rol marcado como es_protegido = true.
Aplicado a	RF-03 (Gestión de roles)
Restricción origen RF	"El rol Administrador es un rol protegido e inmutable, no puede ser eliminado"
Justificación	Es una restricción de integridad estructural. El backend podría tener un bug o ser

	evitado; la BD debe ser la última línea de defensa.

```
CREATE OR REPLACE FUNCTION modulo1.trg_fn_proteger_rol_admin()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' AND OLD.es_protegido = TRUE THEN
        RAISE EXCEPTION 'PROTECTED_ROLE: El rol "%" está marcado como
protegido y no puede ser eliminado.', OLD.nombre_rol
            USING ERRCODE = 'P0004';
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.es_protegido = TRUE AND NEW.nombre_rol
<> OLD.nombre_rol THEN
        RAISE EXCEPTION 'PROTECTED_ROLE: El nombre del rol protegido "%"
no puede ser modificado.', OLD.nombre_rol
            USING ERRCODE = 'P0004';
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.es_protegido = TRUE AND NEW.es_protegido
= FALSE THEN
        RAISE EXCEPTION 'PROTECTED_ROLE: No se puede desmarcar la
protección del rol "%".', OLD.nombre_rol
            USING ERRCODE = 'P0004';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_proteger_rol_admin_update
BEFORE UPDATE ON modulo1.roles
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_proteger_rol_admin();

CREATE TRIGGER trg_proteger_rol_admin_delete
BEFORE DELETE ON modulo1.roles
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_proteger_rol_admin();
```

TRG-06 — Bloqueo de eliminación de rol con usuarios vinculados

Descripción	Impide el DELETE de un rol si existen usuarios actualmente asignados a él.
Aplicado a	RF-03 (Gestión de roles)
Restricción origen RF	"Se prohíbe la eliminación de roles que tengan usuarios vinculados"
Justificación	La FK entre usuarios.id_rol y roles.id_rol ya existe, pero con ON DELETE NO ACTION que genera un error genérico. Este trigger reemplaza ese error con un mensaje de negocio preciso y controlado.

```
CREATE OR REPLACE FUNCTION
modulo1.trg_fn_bloquear_eliminacion_rol_en_uso()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo1.usuarios
    WHERE id_rol = OLD.id_rol;

    IF v_count > 0 THEN
        RAISE EXCEPTION 'ROLE_IN_USE: No se puede eliminar el rol "%"
porque tiene % usuario(s) vinculado(s). Reasigne los usuarios antes de
proceder.', OLD.nombre_rol, v_count
            USING ERRCODE = 'P0005';
    END IF;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bloquear_eliminacion_rol_en_uso
BEFORE DELETE ON modulo1.roles
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_bloquear_eliminacion_rol_en_uso();
```

TRG-07 — Integridad mínima de permisos por rol (no rol huérfano)

Descripción	Impide que un rol quede sin permisos activos. Bloquea tanto la eliminación de un permiso como su desactivación si este es el último permiso activo asociado al rol.
Aplicado a	RF-04 (Gestión de permisos), RF-03 (Gestión de roles)
Restricción origen RF	*"Todo rol debe tener al menos un permiso asociado"* / flujo alterno "Violación de integridad (Rol huérfano)"*
Justificación	Garantía de integridad referencial del modelo RBAC. Un rol sin permisos activos es estructuralmente inválido. Inicialmente, la restricción solo cubría la eliminación de permisos (DELETE), pero existía un vacío cuando un permiso se desactivaba (`es_activo = false`). Esta versión extiende la protección a operaciones de actualización, garantizando la integridad tanto en eliminaciones físicas como en desactivaciones lógicas.

```
CREATE OR REPLACE FUNCTION modulo1.trg_fn_validar_permiso_minimo_rol()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
    v_nombre_rol VARCHAR(100);
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo1.permisos
    WHERE id_rol = OLD.id_rol
    AND id_permiso <> OLD.id_permiso
    AND es_activo = TRUE;

    IF v_count = 0 THEN
        SELECT nombre_rol INTO v_nombre_rol
        FROM modulo1.roles WHERE id_rol = OLD.id_rol;

        RAISE EXCEPTION 'MIN_PERMISSION: No se puede eliminar el permiso
```

```
porque es el último activo del rol "%". Un rol debe tener al menos un
permiso.',
        v_nombre_rol
        USING ERRCODE = 'P0006';
    END IF;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_permiso_minimo_rol
BEFORE DELETE ON modulo1.permisos
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_validar_permiso_minimo_rol();
```

- Validación en desactivación de permisos (UPDATE)

```
CREATE OR REPLACE FUNCTION
modulo1.trg_fn_validar_permiso_minimo_rol_update()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
    v_nombre_rol VARCHAR(100);
BEGIN
    -- Solo validar cuando se intenta desactivar el permiso
    IF OLD.es_activo = TRUE AND NEW.es_activo = FALSE THEN

        SELECT COUNT(*) INTO v_count
        FROM modulo1.permisos
        WHERE id_rol = OLD.id_rol
        AND id_permiso <> OLD.id_permiso
        AND es_activo = TRUE;

        IF v_count = 0 THEN
            SELECT nombre_rol INTO v_nombre_rol
            FROM modulo1.roles
            WHERE id_rol = OLD.id_rol;

            RAISE EXCEPTION 'MIN_PERMISSION: No se puede desactivar el
permiso porque es el último activo del rol "%". Un rol debe tener al
menos un permiso activo.',
            v_nombre_rol
            USING ERRCODE = 'P0006';
        END IF;
```

```
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_permiso_minimo_rol_update
BEFORE UPDATE OF es_activo ON modulo1.permisos
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_validar_permiso_minimo_rol_update();
```

Con esta nueva implementación:

Se evita eliminar el último permiso activo de un rol
Se evita desactivar el último permiso activo de un rol
Se garantiza completamente la regla de integridad del modelo RBAC

TRG-08 — Invalidación de sesiones activas al cambiar estado de cuenta

Descripción	Cuando el estado de una cuenta cambia a INACTIVO, BLOQUEADO o ELIMINADO, marca todas las sesiones activas del usuario como inactivas (es_activa = false).
Aplicado a	RF-06 (Gestión de cuentas), RF-02 (Autenticación)
Restricción origen RF	"Si el estado cambia a INACTIVO, BLOQUEADO o ELIMINADO, el sistema invalidará todos los tokens JWT activos del usuario"
Justificación	La BD puede garantizar que el registro de sesión quede marcado como inactivo. La revocación real del JWT (blacklist en Redis) es responsabilidad del backend, pero la BD registra el estado para que cualquier verificación contra la BD sea consistente.

```
CREATE OR REPLACE FUNCTION
modulo1.trg_fn_invalidar_sesiones_por_estado()
RETURNS TRIGGER AS $$
```

```
DECLARE
    v_estado_nuevo VARCHAR(55);
BEGIN
    SELECT nombre INTO v_estado_nuevo
    FROM modulo1.estados_cuentas
    WHERE id_estado_cuenta = NEW.id_estado_cuenta;

    IF v_estado_nuevo IN ('INACTIVO', 'BLOQUEADO', 'ELIMINADO')
       AND NEW.id_estado_cuenta <> OLD.id_estado_cuenta THEN

        UPDATE modulo1.sesiones
        SET es_activa = FALSE
        WHERE id_cuenta_usuario = NEW.id_cuenta_usuario
          AND es_activa = TRUE;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_invalidar_sesiones_por_estado
AFTER UPDATE OF id_estado_cuenta ON modulo1.cuentas_usuarios
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_invalidar_sesiones_por_estado();
```

TRG-09 — Reset de contador de intentos fallidos al activar cuenta

Descripción	Cuando una cuenta pasa al estado ACTIVO, resetea intentos_fallidos a 0 y limpia bloqueado_hasta.
Aplicado a	RF-02 (Autenticación), RF-06 (Gestión de cuentas)
Restricción origen RF	*"El sistema desbloquea la cuenta después de 15 minutos"* — garantía de consistencia tras reactivación administrativa.
Justificación	Garantía de consistencia del estado. Si el backend activa la cuenta sin limpiar el contador, el usuario queda en un estado inconsistente.

```
CREATE OR REPLACE FUNCTION modulo1.trg_fn_reset_intentos_al_activar()
RETURNS TRIGGER AS $$
DECLARE
    v_estado_nuevo VARCHAR(55);
BEGIN
    SELECT nombre INTO v_estado_nuevo
    FROM modulo1.estados_cuentas
    WHERE id_estado_cuenta = NEW.id_estado_cuenta;

    IF v_estado_nuevo = 'ACTIVO' AND NEW.id_estado_cuenta <>
OLD.id_estado_cuenta THEN
        NEW.intentos_fallidos   := 0;
        NEW.bloqueado_hasta     := NULL;
        NEW.ultimo_intento_fallido := NULL;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_reset_intentos_al_activar
BEFORE UPDATE OF id_estado_cuenta ON modulo1.cuentas_usuarios
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_reset_intentos_al_activar();
```

TRG-10 — Protección de campos críticos del usuario (id_usuario no modificable)

Descripción	Impide la modificación del id_usuario y del numero_identificacion una vez registrados.
Aplicado a	RF-05 (Edición de datos)
Restricción origen RF	"El identificador del usuario (id_usuario) no puede ser modificado"
Justificación	Es la integridad referencial más básica. El PK no debería cambiar nunca; este trigger lo garantiza incluso si el backend lo intenta por error.

```
CREATE OR REPLACE FUNCTION
modulo1.trg_fn_proteger_campos_criticos_usuario()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.id_usuario <> OLD.id_usuario THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: El campo id_usuario es
inmutable y no puede ser modificado.'
            USING ERRCODE = 'P0007';
    END IF;

    IF NEW.numero_identificacion <> OLD.numero_identificacion THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: El número de identificación no
puede ser modificado una vez registrado.'
            USING ERRCODE = 'P0007';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_proteger_campos_criticos_usuario
BEFORE UPDATE ON modulo1.usuarios
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_proteger_campos_criticos_usuario();
```

TRG-11 — Token de un solo uso (one-time use)

Descripción	Cuando fecha_uso se establece en un token, verifica que no haya sido usado previamente (que fecha_uso fuera NULL antes del UPDATE).
Aplicado a	RF-08 (Recuperación), RF-09 (Restablecimiento), RF-01 (Activación)
Restricción origen RF	*"No se deben reutilizar tokens" / "unique, de un solo uso (one-time use)"*
Justificación	El campo fecha_uso en la tabla tokens es la fuente de verdad del uso del token. Este trigger garantiza que una vez marcado como usado, no pueda "desmarcarse" ni usarse de

	nuevo.

```
CREATE OR REPLACE FUNCTION modulo1.trg_fn_token_un_solo_uso()
RETURNS TRIGGER AS $$
BEGIN
    -- Si ya tenía fecha_uso (ya fue usado), bloquear cualquier nuevo
uso
    IF OLD.fecha_uso IS NOT NULL AND NEW.fecha_uso IS NOT NULL
       AND NEW.fecha_uso <> OLD.fecha_uso THEN
        RAISE EXCEPTION 'TOKEN_ALREADY_USED: Este token ya fue utilizado
el %. No puede reutilizarse.', OLD.fecha_uso
            USING ERRCODE = 'P0008';
    END IF;

    -- No se puede "desmarcar" un token usado
    IF OLD.fecha_uso IS NOT NULL AND NEW.fecha_uso IS NULL THEN
        RAISE EXCEPTION 'IMMUTABLE_FIELD: El estado de uso de un token
no puede revertirse.'
            USING ERRCODE = 'P0008';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_token_un_solo_uso
BEFORE UPDATE OF fecha_uso ON modulo1.tokens
FOR EACH ROW
EXECUTE FUNCTION modulo1.trg_fn_token_un_solo_uso();
```